# Experiment 12: CNN Architecture and Hyperparameter Comparison

## Question

**Problem Statement:** The recycling plant's CNN must run on a low-cost edge device. The engineer must balance accuracy against model size and training time. The same waste or CIFAR-10 dataset from Experiment 11. The objective is to compare CNN architectures and hyperparameters systematically. Use TensorFlow, Keras, NumPy, Matplotlib.

**Tasks:**

1. Vary the number of convolution layers (2, 3, 4).
2. Vary filter size (3×3, 5×5) and filter count.
3. Compare max pooling and average pooling, with and without batch normalisation.
4. Vary learning rate and batch size.
5. Tabulate accuracy, parameter count and training time for each configuration.

## Code

```
"""Experiment 12: CNN Architecture and Hyperparameter Comparison (CIFAR-10)"""
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42); tf.random.set_seed(42)
USE_SYNTHETIC = False        # keep False; True only generates random images to smoke-test the code
EPOCHS = 6                   # kept small so that all configurations finish quickly
N_TRAIN = 20000              # training subset size (reduce if your machine is slow)

# ---------------- Data ----------------
if USE_SYNTHETIC:
    x_train = np.random.rand(1500, 32, 32, 3).astype("float32"); y_train = np.random.randint(0, 10, 1500)
    x_test = np.random.rand(500, 32, 32, 3).astype("float32"); y_test = np.random.randint(0, 10, 500)
else:
    (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
    x_train = x_train[:N_TRAIN].astype("float32") / 255.0; y_train = y_train[:N_TRAIN].ravel()
    x_test = x_test.astype("float32") / 255.0; y_test = y_test.ravel()

# ---------------- Model builder ----------------
def build_cnn(n_conv=2, kernel=3, filters=32, pool="max", bn=False, lr=1e-3):
    m = keras.Sequential([layers.Input((32, 32, 3))])
    for i in range(n_conv):
        m.add(layers.Conv2D(filters * (2 ** i), kernel, padding="same"))
        if bn:
            m.add(layers.BatchNormalization())
        m.add(layers.Activation("relu"))
        m.add(layers.MaxPooling2D(2) if pool == "max" else layers.AveragePooling2D(2))
    m.add(layers.Flatten())
    m.add(layers.Dense(64, activation="relu"))
    m.add(layers.Dense(10, activation="softmax"))
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    return m

BASE = dict(n_conv=2, kernel=3, filters=32, pool="max", bn=False, lr=1e-3, batch=64)

# One-factor-at-a-time experiments (Tasks 1-4)
experiments = [
    ("Baseline", {}),
    # Task 1: number of conv layers
    ("3 conv layers", dict(n_conv=3)), ("4 conv layers", dict(n_conv=4)),
    # Task 2: filter size and count
    ("5x5 filters", dict(kernel=5)), ("16 base filters", dict(filters=16)), ("64 base filters",
dict(filters=64)),
    # Task 3: pooling and batch normalisation
```

```
    ("Avg pooling", dict(pool="avg")), ("Max pool + BN", dict(bn=True)), ("Avg pool + BN", dict(pool="avg",
bn=True)),
    # Task 4: learning rate and batch size
    ("lr = 0.01", dict(lr=1e-2)), ("lr = 0.0001", dict(lr=1e-4)),
    ("batch = 32", dict(batch=32)), ("batch = 128", dict(batch=128)),
]

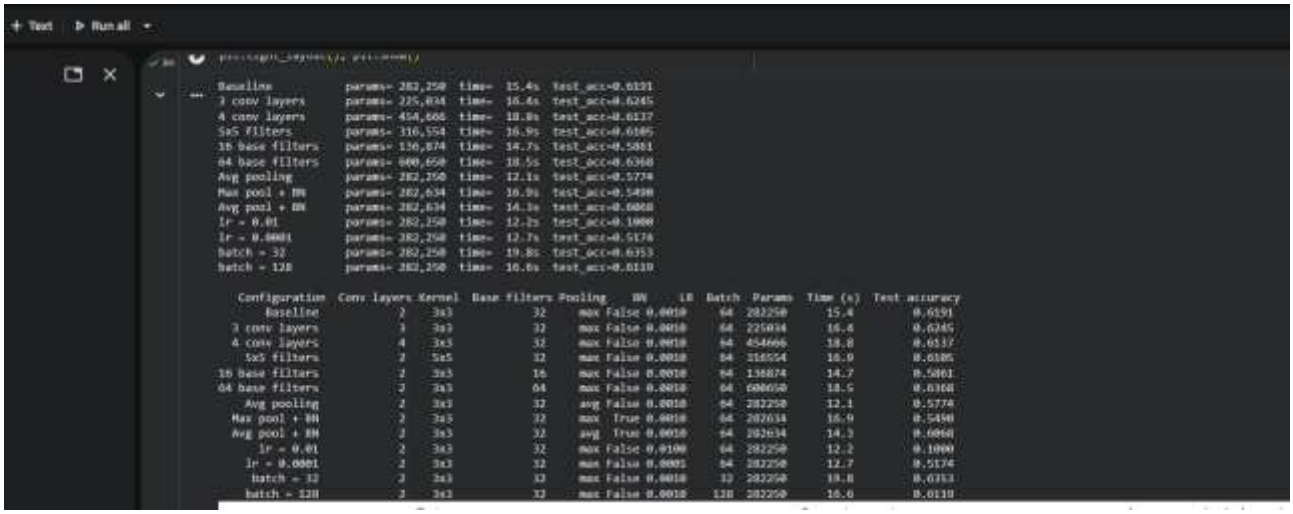
rows = []
for name, override in experiments:
    cfg = {**BASE, **override}
    batch = cfg.pop("batch")
    model = build_cnn(**cfg)
    t0 = time.time()
    model.fit(x_train, y_train, epochs=EPOCHS, batch_size=batch, validation_split=0.1, verbose=0)
    train_time = time.time() - t0
    test_acc = model.evaluate(x_test, y_test, verbose=0)[1]
    rows.append({"Configuration": name, "Conv layers": cfg["n_conv"], "Kernel":
f"{cfg['kernel']}x{cfg['kernel']}",
                 "Base filters": cfg["filters"], "Pooling": cfg["pool"], "BN": cfg["bn"], "LR": cfg["lr"],
                 "Batch": batch, "Params": model.count_params(), "Time (s)": round(train_time, 1),
                 "Test accuracy": round(test_acc, 4)})
    print(f"{name:18} params={model.count_params():>8,}  time={train_time:6.1f}s  test_acc={test_acc:.4f}")

# ---------------- Task 5: results table ----------------
df = pd.DataFrame(rows)
pd.set_option("display.width", 200); pd.set_option("display.max_columns", None)
print("\n", df.to_string(index=False))
df.to_csv("cnn_comparison_results.csv", index=False)

fig, ax = plt.subplots(1, 3, figsize=(18, 5))
ax[0].barh(df["Configuration"], df["Test accuracy"]); ax[0].set_title("Test accuracy"); ax[0].invert_yaxis()
ax[1].barh(df["Configuration"], df["Params"]); ax[1].set_title("Parameter count"); ax[1].invert_yaxis()
ax[2].scatter(df["Params"], df["Test accuracy"], c=df["Time (s)"], cmap="viridis", s=80)
for _, r in df.iterrows():
    ax[2].annotate(r["Configuration"], (r["Params"], r["Test accuracy"]), fontsize=7)
ax[2].set_xlabel("Parameters"); ax[2].set_ylabel("Test accuracy"); ax[2].set_title("Accuracy vs size (colour
= training time)")
plt.tight_layout(); plt.show()
```

## Output



```
Baseline            params= 282,250  time=  15.4s  test_acc=0.6191
3 conv layers       params= 225,034  time=  16.6s  test_acc=0.6245
4 conv layers       params= 454,666  time=  18.8s  test_acc=0.6137
5x5 filters         params= 316,554  time=  16.9s  test_acc=0.6385
16 base filters     params= 138,874  time=  14.7s  test_acc=0.5861
64 base filters     params= 600,650  time=  18.5s  test_acc=0.6368
Avg pooling         params= 282,250  time=  12.1s  test_acc=0.5774
Max pool + BN       params= 282,634  time=  16.9s  test_acc=0.5490
Avg pool + BN       params= 282,634  time=  14.3s  test_acc=0.6868
lr = 0.01           params= 282,250  time=  12.2s  test_acc=0.1000
lr = 0.0001         params= 282,250  time=  12.7s  test_acc=0.5174
batch = 32          params= 282,250  time=  19.8s  test_acc=0.6353
batch = 128         params= 282,250  time=  16.6s  test_acc=0.6110

    Configuration  Conv layers Kernel  Base filters Pooling   BN     LR  Batch  Params  Time (s)  Test accuracy
         Baseline            2    3x3           32     max False 0.0010     64  282250      15.4         0.6191
  3 conv layers              3    3x3           32     max False 0.0010     64  225034      16.6         0.6245
  4 conv layers              4    3x3           32     max False 0.0010     64  454666      18.8         0.6137
    5x5 filters              2    5x5           32     max False 0.0010     64  316554      16.9         0.6385
16 base filters              2    3x3           16     max False 0.0010     64  138874      14.7         0.5861
64 base filters              2    3x3           64     max False 0.0010     64  600650      18.5         0.6368
    Avg pooling              2    3x3           32     avg False 0.0010     64  282250      12.1         0.5774
  Max pool + BN              2    3x3           32     max  True 0.0010     64  282634      16.9         0.5490
  Avg pool + BN              2    3x3           32     avg  True 0.0010     64  282634      14.3         0.6868
       lr = 0.01             2    3x3           32     max False 0.0100     64  282250      12.2         0.1000
     lr = 0.0001             2    3x3           32     max False 0.0001     64  282250      12.7         0.5174
      batch = 32             2    3x3           32     max False 0.0010     32  282250      19.8         0.6353
     batch = 128             2    3x3           32     max False 0.0010    128  282250      16.6         0.6110
```